

Verilog HDL Code for Basic **Sequential Logic**

Prepared by Dr. Shahriyar Masud Rizvi

Behavioral Coding Style in Verilog HDL

- At behavioral level a digital device is described in terms of its behavior, that is its function or algorithm, rather than its structure. In other words, in behavioral style, we model what a device does, rather than what parts it is made of.
- Behavioral style allows a very concise description of hardware. Without behavioral style, one cannot describe practical devices, as the codes written at structural (gate-level) level may be 10x to 100x longer as compared to their behavioral versions.
- Behavioral code must always be contained inside *always* blocks.
- Verilog has two categories of data types-nets and variables. The *wire* is an example of a net. The *reg* is an example of a variable.
- Output of *always* blocks must be defined as a *reg* type variable.

```
module D_FF (input wire CLK, RST, D,  
             output reg Q);  
  
wire NEXT_Q;  
  
assign NEXT_Q = ~RST & D;  
  
always @ (posedge CLK)  
begin  
    Q <= NEXT_Q;  
end
```

- Behavioral code allows control structures such as *if*, *case* etc. to be used to describe a device.

```
always @ (posedge CLK)  
begin  
    if (RST) Q <= 0;  
    else  
        case ({J,K})  
            2'b00: Q <= Q;  
            2'b01: Q <= 0;  
            2'b10: Q <= 1;  
            2'b11: Q <= ~Q;  
            default: ;  
        endcase  
end
```

- The *always* blocks require a sensitivity list-it is a list of signals the *always* block is sensitive to.
- For combinational logic, sensitivity list should include any signal that is being evaluated (by an *if*, *case* etc. control structures) or operated on (that means any signal that is on the right-hand side of assignments (=)).
- For sequential logic, typically only clock signal should be included in the sensitivity list. The clock signal should be preceded by either *posedge* or *negedge*. If there is any asynchronous input, it should be included in the sensitivity list.

```
always @ (posedge CLK)
begin
```

```
if (RST) Q <= 0;
else
```

- +ve edge-triggered sequential logic must utilize *posedge* to detect rising edge of the clock. -ve edge-triggered sequential logic must utilize *negedge* to detect falling edge of the clock.
- Statements inside *always* blocks for combinational logic must be assigned using blocking assignment “=”. Blocking assignments operate serially (one after another).
- Statements inside *always* blocks for sequential logic must be assigned using non-blocking assignment “<=”. Non-blocking assignments operate concurrently (parallelly). So, order of statements does not matter here.

```
always @ (posedge CLK)
begin
```

```
if (RST) Q <= 0;
else
```

- Note that *always* blocks operate in parallel. So, the order of *always* blocks does not matter.
- **In this course note, we will be utilizing behavioral code only for the present state flip-flops. For the rest of the circuit (Next_Q logic), structural code will be utilized. So, the resulting code is structural code with a behavioral segment.**

```
assign NEXT_Q = ~RST & D;
always @ (posedge CLK)
begin
    Q <= NEXT_Q;
end
```

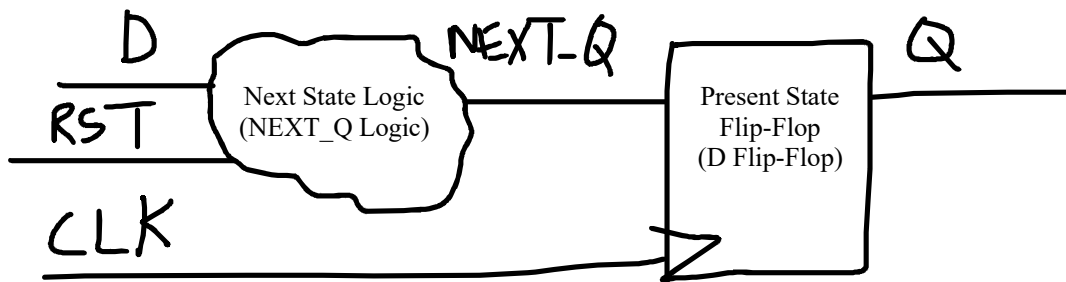
Structural Style

Behavioral Style

D Flip-Flop

+ve edge triggered D Flip-Flop with synchronous reset signal (RST).

Architecture



State Table

RST	NEXT_Q
0	D
1	0

CLK	Q
↑	NEXT_Q

Next_Q Equation

$$NEXT_Q = \overline{RST} \cdot D$$

Verilog HDL code

```
module D_FF (input wire CLK, RST, D,  
             output reg Q);
```

```
wire NEXT_Q;
```

```
assign NEXT_Q = ~RST & D; ← Structural Style
```

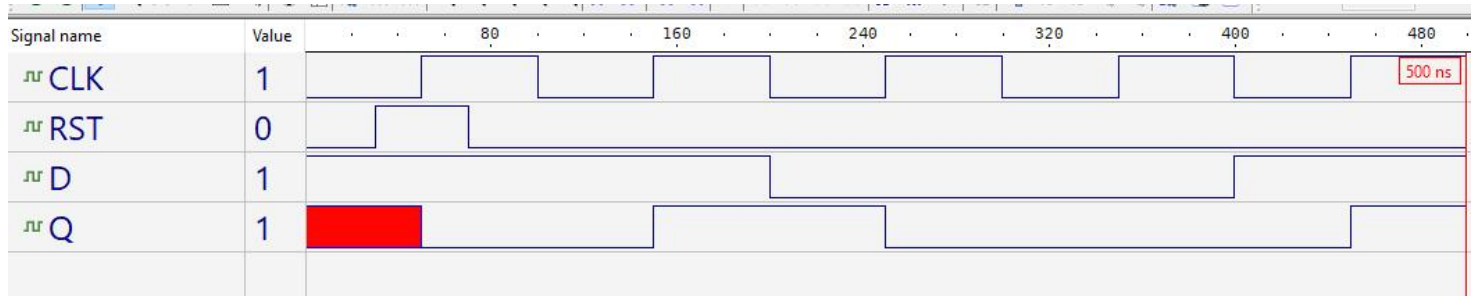
```
always @ (posedge CLK)  
begin ← Behavioral Style
```

```
    Q <= NEXT_Q;
```

```
end
```

```
endmodule
```

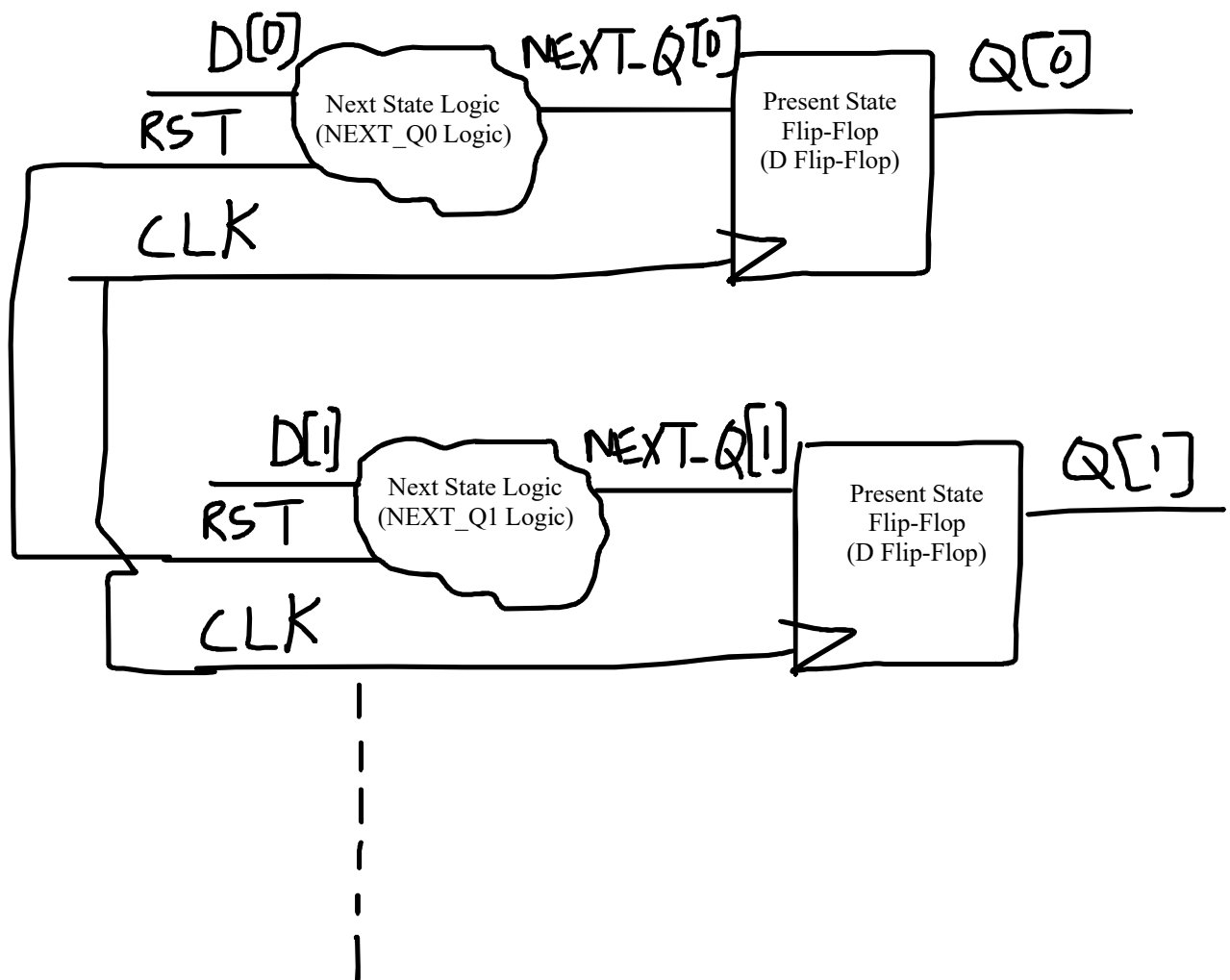
Functional Simulation



D Register

+ve edge triggered 4-bit D register with synchronous reset signal (RST).

Architecture



State Table

RST	NEXT_Q0	NEXT_Q1	NEXT_Q2	NEXT_Q3
0	D0	D1	D2	D3
1	0	0	0	0

CLK	Q0	Q1	Q2	Q3
$\uparrow$	NEXT_Q0	NEXT_Q1	NEXT_Q2	NEXT_Q3

Next_Q Equation

$$NEXT_Q0 = \overline{RST}.D0 \quad NEXT_Q1 = \overline{RST}.D1 \quad NEXT_Q2 = \overline{RST}.D2 \quad NEXT_Q3 = \overline{RST}.D3$$

Verilog HDL Code:

```
module D_REG_4BIT (input wire CLK, RST,
                  input wire [3:0] D,
                  output reg [3:0] Q);
```

```
wire NEXT_Q[3:0];
```

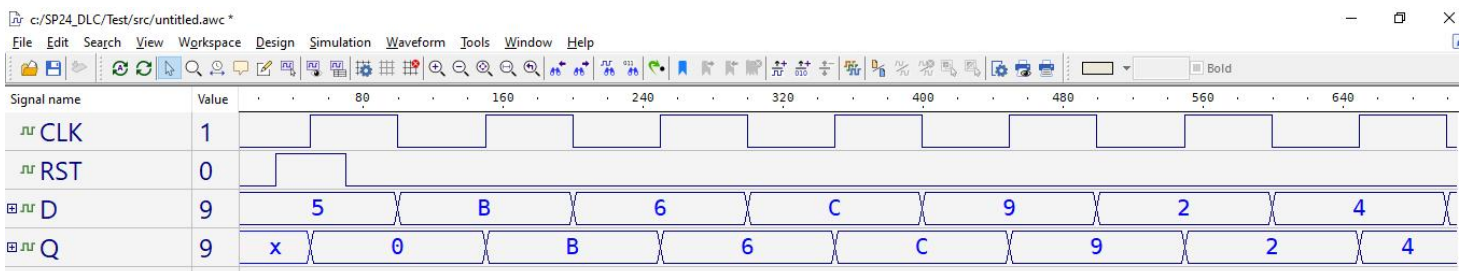
```
assign NEXT_Q[0] = ~RST & D[0];
assign NEXT_Q[1] = ~RST & D[1];
assign NEXT_Q[2] = ~RST & D[2];
assign NEXT_Q[3] = ~RST & D[3];
```

```
always @ (posedge CLK)
begin
```

```
Q[0] <= NEXT_Q[0];
Q[1] <= NEXT_Q[1];
Q[2] <= NEXT_Q[2];
Q[3] <= NEXT_Q[3];
```

```
end
endmodule
```

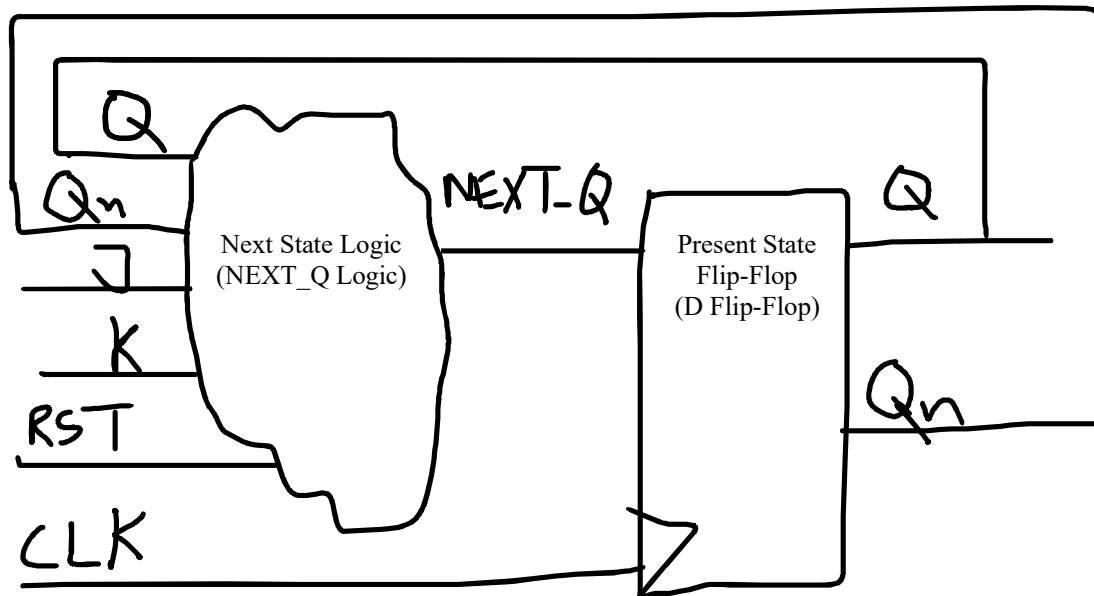
Functional Simulation



J-K Flip-Flop

+ve edge triggered J-K Flip-Flop with synchronous reset signal (RST).

Architecture



State Table

RST	J	K	NEXT_Q
1	x	x	Q
0	0	0	Q
0	0	1	0
0	1	0	1
0	1	1	$\bar{Q}$

CLK	Q	Qn
$\uparrow$	NEXT_Q	$\overline{NEXT_Q}$

Next_Q Equation

$$NEXT_Q = \overline{RST}(\bar{J}.\bar{K}.Q + J.\bar{K} + J.K.Qn)$$

Verilog HDL Code

```
module JK_FF_RST
  (input wire CLK, RST, J, K,
   output reg Q, Qn);
  wire NEXT_Q;

  assign NEXT_Q = (~RST & ~J & ~K & Q) | (~RST & J & ~K) | (~RST & J & K & Qn);

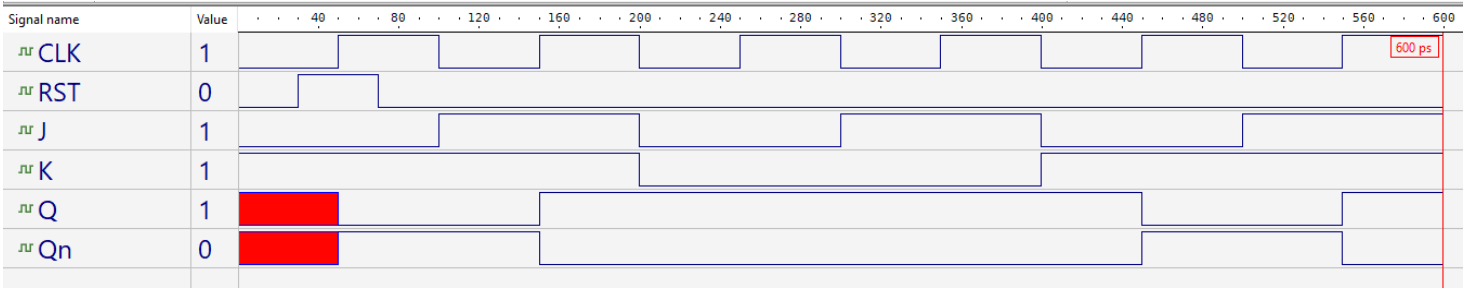
  always @ (posedge CLK)
  begin

    Q <= NEXT_Q;
    Qn <= ~NEXT_Q;

  end

endmodule
```

Functional Simulation



For Interested Readers

Pure Behavioral (industry-standard) coding style for sequential logic.

These approaches do not require developing truth table, state table and logic equations. The devices are described algorithmically.

<u>Structural Style with some behavioral segment (D Flip-Flop)</u>	<u>Pure behavioral style (D Flip-Flop)</u>
<pre><i>module</i> D_FF (<i>input wire</i> CLK, RST, D, <i>output reg</i> Q); <i>wire</i> NEXT_Q; <i>assign</i> NEXT_Q = ~RST & D; <i>always @ (posedge CLK)</i> <i>begin</i> Q <= NEXT_Q; <i>end</i> <i>endmodule</i></pre>	<pre><i>module</i> D_FF (<i>input wire</i> CLK, RST, D, <i>output reg</i> Q); <i>always @ (posedge CLK)</i> <i>begin</i> <i>if</i> (RST) Q <= 0; <i>else</i> Q <= D; <i>end</i> <i>endmodule</i></pre>
<u>Structural Style with some behavioral segment (D Register)</u>	<u>Pure behavioral style (D Register)</u>
<pre><i>module</i> D_REG_4BIT (<i>input wire</i> CLK, RST, <i>input wire</i> [3:0] D, <i>output reg</i> [3:0] Q); <i>wire</i> NEXT_Q[3:0]; <i>assign</i> NEXT_Q[0] = ~RST & D[0]; <i>assign</i> NEXT_Q[1] = ~RST & D[1]; <i>assign</i> NEXT_Q[2] = ~RST & D[2]; <i>assign</i> NEXT_Q[3] = ~RST & D[3]; <i>always @ (posedge CLK)</i> <i>begin</i> Q[0] <= NEXT_Q[0]; Q[1] <= NEXT_Q[1]; Q[2] <= NEXT_Q[2]; Q[3] <= NEXT_Q[3]; <i>end</i> <i>endmodule</i></pre>	<pre><i>module</i> D_REG_4BIT (<i>input wire</i> CLK, RST, <i>input wire</i> [3:0] D, <i>output reg</i> [3:0] Q); <i>always @ (posedge CLK)</i> <i>begin</i> <i>if</i> (RST) Q <= 0; <i>else</i> Q <= D; <i>end</i> <i>endmodule</i></pre>

<u>Structural Style with some behavioral segment (J-K Flip-Flop)</u>	<u>Pure behavioral style (J-K Flip-Flop)</u>
<pre> <i>module</i> JK_FF_RST (<i>input wire</i> CLK, RST, J, K, <i>output reg</i> Q, Qn); <i>wire</i> NEXT_Q; <i>assign</i> NEXT_Q = (~RST & ~J & ~K & Q) (~RST & J & ~K) (~RST & J & K & Qn); <i>always @ (posedge CLK)</i> <i>begin</i> Q <= NEXT_Q; Qn <= ~NEXT_Q; <i>end</i> <i>endmodule</i> </pre>	<pre> <i>module</i> JK_FF_RST (<i>input wire</i> CLK, RST, J, K, <i>output reg</i> Q, Qn); <i>always @ (posedge CLK)</i> <i>begin</i> <i>if</i> (RST) Q <= 0; <i>else</i> <i>case</i> ({J,K}) 2'b00: Q <= Q; 2'b01: Q <= 0; 2'b10: Q <= 1; 2'b11: Q <= ~Q; <i>default</i>: ; <i>endcase</i> <i>end</i> <i>always @ (Q)</i> <i>begin</i> Qn <= ~Q; <i>end</i> <i>endmodule</i> </pre>

Reference:

1. Michael Ciletti, Advanced Digital Design with the Verilog HDL-2nd Edition, 2010.
2. Douglas J. Smith, HDL Chip Design-A Practical Guide for Designing, Synthesizing and Simulating ASICs & FPGAs using VHDL or Verilog, 1997.